

Wipro - 100 Technical Interview Questions

Wipro Limited

With Answers & Diagrams - For BTech Computer Science / IT Students

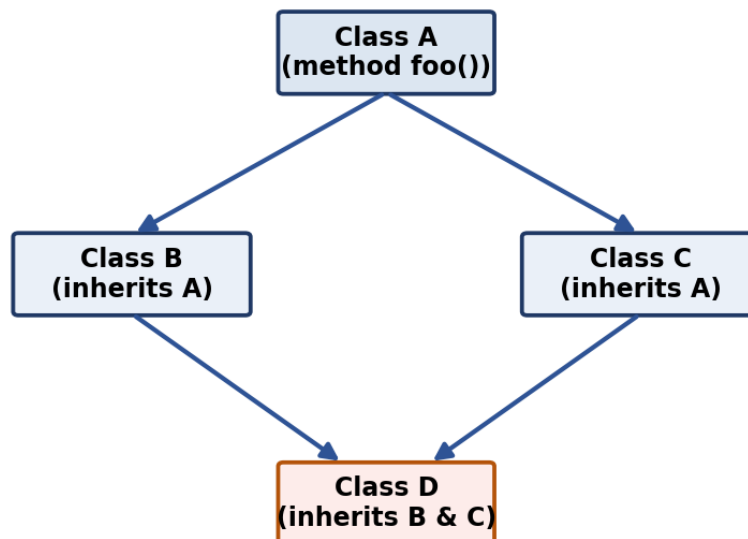
Object-Oriented Programming

Q1. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.

Diamond Problem: which foo() does D inherit - via B or via C?



Q2. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q3. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q4. What is method hiding, and how is it different from method overriding?

Answer:

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its parent - the parent's version isn't overridden, just hidden, and which one runs depends on the reference type used to call it (resolved at compile time). Method overriding, in contrast, applies to instance methods and is resolved at runtime based on the actual object type.

Q5. Explain the difference between an abstract class and an interface. When would you choose one over the other?

Answer:

An abstract class can mix abstract and concrete methods, hold state (fields), and a class can extend only one abstract class. An interface (pre-Java 8) only declares method signatures with no implementation, and a class can implement many interfaces. Use an abstract class when subclasses share common state/behaviour; use an interface when you only need to guarantee a contract across unrelated classes.

Q6. What is constructor overloading? Can constructors be inherited?

Answer:

Constructor overloading means a class has multiple constructors with different parameter lists, letting objects be created in different ways (e.g. Point() vs Point(x,y)). Constructors are NOT inherited by subclasses - a subclass must define its own constructors, though it can call a parent constructor explicitly via super().

Q7. Explain encapsulation and why it is important for maintainable software.

Answer:

Encapsulation hides an object's internal state behind private fields and exposes controlled access through public getters/setters or methods. It matters because it lets you change internal implementation without breaking external code, enforce validation (e.g. reject a negative balance), and reduces unintended coupling between modules.

Q8. What design pattern would you use to ensure only a single instance of a class exists, and how would you implement it thread-safely?

Answer:

The Singleton design pattern ensures only one instance of a class exists. A thread-safe implementation typically uses either double-checked locking with a volatile instance field, an eagerly-initialized static final instance, or (in Java) a single-element enum, all of which avoid two threads creating two separate instances under concurrent access.

Q9. Describe a scenario from your own project where you used inheritance, and explain why it was the right choice.

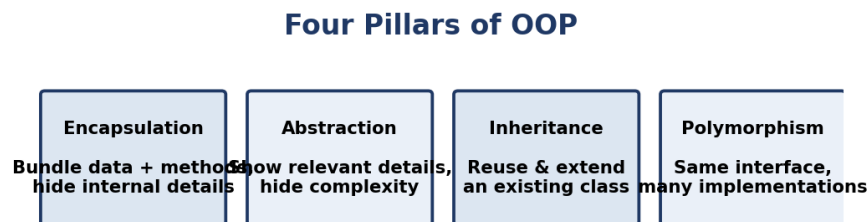
Answer:

Example answer structure: describe a project where a base class captured shared behaviour (e.g. a base Notification class with a send() method) and subclasses (EmailNotification, SMSNotification) overrode send() with channel-specific logic - explain why this avoided duplicating shared validation/logging code and made adding a new channel a matter of adding one new subclass.

Q10. Explain the four pillars of Object-Oriented Programming with a real-world example for each.

Answer:

The four pillars are: **Encapsulation** - bundling data and methods together and hiding internal state (e.g. a bank account class exposing deposit()/withdraw() but hiding the raw balance field); **Abstraction** - exposing only relevant details, like a car's steering wheel hiding the mechanics underneath; **Inheritance** - a Manager class reusing and extending an Employee class; **Polymorphism** - a draw() method behaving differently for a Circle vs a Square object called through a common Shape reference.



Q11. What is polymorphism? Differentiate between compile-time and runtime polymorphism with examples.

Answer:

Polymorphism lets the same interface behave differently depending on the actual object. Compile-time (static) polymorphism is resolved during compilation - method/operator overloading is the classic example. Runtime (dynamic) polymorphism is resolved during execution based on the actual object type - method overriding via a base-class reference is the classic example.

Q12. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. add(int,int) and add(double,double). Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. Dog overriding Animal's makeSound().

Q13. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q14. Explain the Open/Closed Principle with a practical example.

Answer:

The Open/Closed Principle says classes should be open for extension but closed for modification. Practical example: instead of adding an if-else chain to a `PaymentProcessor` class every time a new payment method is added, define a `PaymentMethod` interface and add new payment types as new classes implementing it - existing code doesn't need to change.

Q15. Explain the concept of a virtual function in C++ and how it enables runtime polymorphism.

Answer:

A virtual function in C++ is declared with the 'virtual' keyword in a base class, telling the compiler to resolve the call at runtime via a vtable based on the actual object type rather than the reference/pointer type. This is what allows a `Shape*` pointing to a `Circle` to correctly call `Circle's draw()` instead of `Shape's` - the mechanism behind runtime polymorphism in C++.

Q16. What is an object in OOP, and how is it different from a class?

Answer:

An object is a concrete instance of a class, with its own state (field values) stored in memory, while a class is the blueprint/template that defines what fields and methods its objects will have. 'Car' is the class; 'myRedCar' is an object of that class.

DBMS & SQL

Q17. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q18. Write a SQL query to find the Nth highest salary using a window function like `RANK()` or `DENSE_RANK()`.

Answer:

```
SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM Employee) ranked WHERE rnk = N (substituting the desired N). DENSE_RANK() assigns ranks without
gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.
```

Q19. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q20. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q21. Explain the ACID properties of a database transaction with an example for each.

Answer:

ACID: **Atomicity** - a transaction's operations either all succeed or all fail together (e.g. a bank transfer's debit and credit both happen or neither does); **Consistency** - a transaction moves the database from one valid state to another, respecting constraints; **Isolation** - concurrent transactions don't see each other's incomplete/intermediate changes; **Durability** - once committed, changes survive a crash (written to persistent storage).

Q22. Explain the difference between a clustered and a non-clustered index.

Answer:

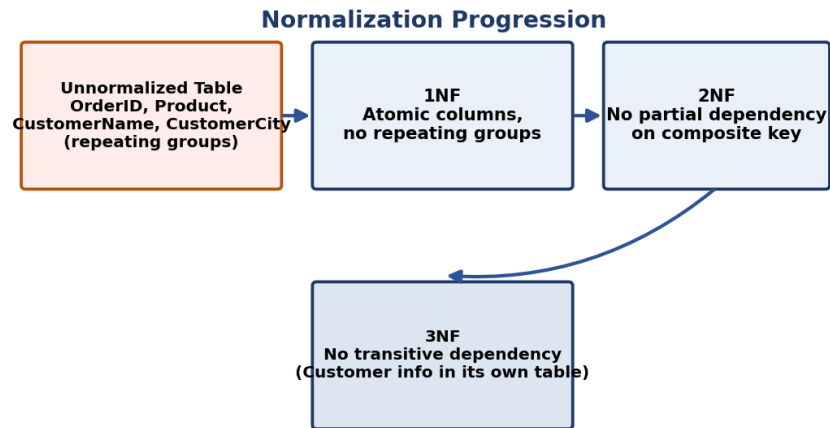
A clustered index determines the physical storage order of the table's data rows - a table can have only one, since data can only be sorted one way physically. A non-clustered index is a separate structure that stores pointers back to the actual data rows, so a table can have several, useful for speeding up queries on columns other than the clustering key.

Q23. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with a practical example.

Answer:

Normalization removes redundancy step by step: **1NF** requires atomic column values with no repeating groups; **2NF** requires 1NF plus no partial dependency of a non-key column on part of a composite primary key; **3NF** requires 2NF plus no transitive dependency (a non-key column depending on another non-key column); **BCNF** is a stricter version of 3NF handling certain anomalies where a table has multiple overlapping candidate keys.

Example: splitting an Orders table that repeats CustomerCity for every order into a separate Customers table removes the transitive dependency.



Q24. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q25. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q26. Write a SQL query to find the second-highest salary from an Employee table without using LIMIT/TOP.

Answer:

Using a subquery: `SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee)`. This first finds the overall maximum, then finds the maximum salary that is strictly less than it, giving the second-highest value without relying on database-specific LIMIT/TOP syntax.

Q27. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q28. What is the difference between DELETE, DROP, and TRUNCATE?**Answer:**

DELETE removes rows one at a time (can be filtered with WHERE, is logged, and can be rolled back) and fires triggers; DROP removes the entire table structure and its data permanently; TRUNCATE quickly removes all rows by deallocating data pages (minimal logging), resets identity counters, but keeps the table structure - and cannot be rolled back in most databases once committed.

Q29. Explain the difference between horizontal and vertical partitioning of a table.**Answer:**

Horizontal partitioning splits a table by rows (e.g. orders from 2023 in one partition, 2024 in another), similar in spirit to sharding but often within a single database instance. Vertical partitioning splits a table by columns (e.g. splitting rarely used large text/blob columns into a separate table), reducing the size of rows scanned for common queries that don't need those columns.

Q30. Write a SQL query to find duplicate records in a table.**Answer:**

Using GROUP BY with HAVING COUNT(*) > 1: `SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name HAVING COUNT(*) > 1`. This groups rows by the column(s) that define a duplicate and returns only the groups appearing more than once.

Q31. How does a B-Tree index improve query performance internally?**Answer:**

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically $O(\log n)$) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q32. What is database replication, and what is the difference between synchronous and asynchronous replication?**Answer:**

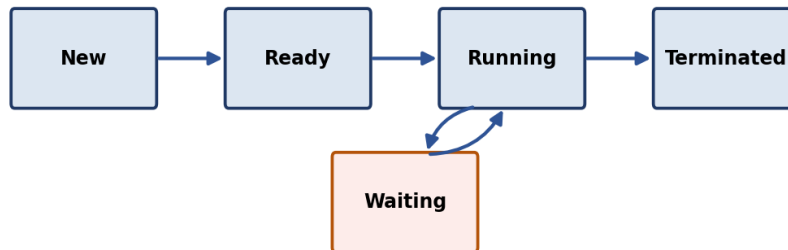
Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Operating Systems

Q33. What is the difference between a process and a thread?**Answer:**

A process is an independent program in execution with its own memory space (code, data, heap, stack) and OS-allocated resources. A thread is a lightweight unit of execution within a process that shares the process's memory and resources with other threads of the same process, making thread creation and context switching cheaper than process creation.

Process State Lifecycle



Q34. Explain the difference between preemptive and non-preemptive scheduling with examples of each.

Answer:

Preemptive scheduling allows the OS to forcibly suspend a running process to give the CPU to another (e.g. Round Robin, Priority Scheduling with preemption) - responsive but has context-switch overhead. Non-preemptive scheduling lets a running process keep the CPU until it voluntarily yields or finishes (e.g. FCFS, non-preemptive SJF) - simpler but a long process can block shorter ones ('convoy effect').

Q35. Explain the difference between paging and segmentation as memory management techniques.

Answer:

Paging divides both physical and logical memory into fixed-size blocks (pages/frames), eliminating external fragmentation but potentially causing internal fragmentation within a page. Segmentation divides a program into variable-sized logical units (code, stack, heap segments) that map more naturally to how programmers think about memory, but can suffer external fragmentation since segments vary in size.



Q36. Describe a real-world analogy to explain deadlock to a non-technical person.

Answer:

Analogy: imagine four cars at a four-way intersection, each waiting for the car to its right to move first before it can proceed - if all four wait simultaneously, none will ever move, even though each car is only blocked because of the others' identical behavior. That's a deadlock: everyone is waiting for someone else to act first, and no one ever will.

Q37. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Q38. What is the Banker's Algorithm, and how does it help avoid deadlocks?

Answer:

The Banker's Algorithm avoids deadlock by only granting a resource request if the resulting state is 'safe' - meaning there exists some order in which all processes could still finish given the maximum resources they might request. It simulates allocation before actually granting it, refusing requests that could lead to an unsafe (potentially deadlocked) state.

Q39. How does an operating system ensure fair CPU allocation among multiple processes?

Answer:

The OS ensures fair CPU allocation through its scheduler, using algorithms like Round Robin (equal time slices for each process) or priority-based scheduling with aging (gradually increasing the priority of processes that have waited a long time, to prevent starvation), combined with preemption so no single process can monopolize the CPU indefinitely.

Q40. Explain what an inode is and what information it stores.

Answer:

An inode is a data structure in Unix-like file systems that stores a file's metadata: permissions, owner, size, timestamps, and pointers to the disk blocks holding the actual file data - notably, it does not store the file's name (that mapping lives in the containing directory), which is why hard links to the same inode can have different names.

Q41. Explain the producer-consumer problem and how you would solve it using semaphores.

Answer:

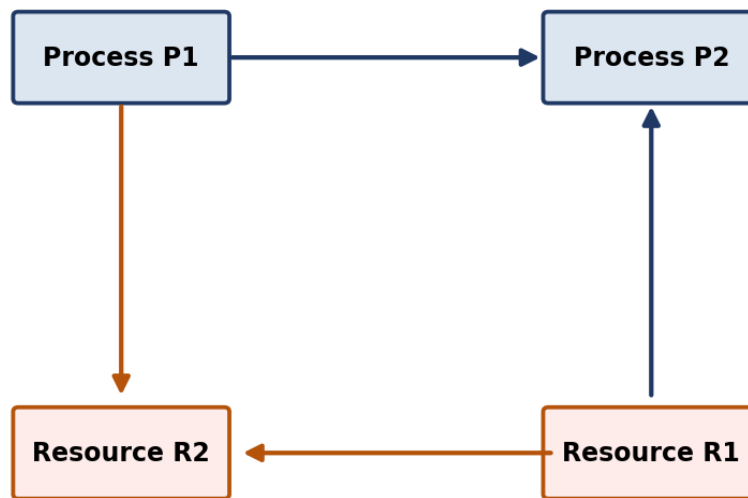
The producer-consumer problem involves a producer adding items to a shared bounded buffer and a consumer removing them, needing synchronization so the producer doesn't add to a full buffer and the consumer doesn't remove from an empty one. It's solved using two counting semaphores (one tracking empty slots, one tracking filled slots) plus a mutex protecting the buffer itself during actual insert/remove operations.

Q42. Explain the four necessary conditions for a deadlock to occur.

Answer:

The four necessary conditions are: **Mutual Exclusion** - at least one resource is held in a non-shareable mode; **Hold and Wait** - a process holds one resource while waiting for another; **No Preemption** - resources can't be forcibly taken away, only released voluntarily; **Circular Wait** - a closed chain of processes each waiting for a resource held by the next. All four must hold simultaneously for a deadlock to occur.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q43. What is a race condition, and how can synchronization primitives like semaphores or mutexes prevent it?

Answer:

A race condition occurs when multiple threads/processes access shared data concurrently and the final result depends on the unpredictable timing/order of execution, potentially producing incorrect results. Synchronization primitives like mutexes (mutual exclusion locks) or semaphores prevent this by ensuring only one thread can enter a critical section that modifies shared data at a time.

Q44. What is virtual memory, and why is it useful even when physical RAM is limited?

Answer:

Virtual memory gives each process the illusion of a large, contiguous address space by mapping logical addresses to physical RAM or disk (swap space) via page tables, transparently to the program. It's useful even with limited RAM because it allows programs larger than physical memory to run, isolates each process's memory from others for safety, and lets the OS overcommit memory across many processes that don't all need their full memory at once.

Q45. What is spooling, and how does it help manage I/O devices like printers?

Answer:

Spooling (Simultaneous Peripheral Operations On-Line) buffers data intended for a slow I/O device (like a printer) on disk, letting a process 'print' instantly to the spool without waiting for the physical device, while a separate spooler process feeds jobs to the device at its own pace - allowing multiple processes to share one device without conflict.

Q46. What is the difference between multiprogramming, multitasking, and multiprocessing?

Answer:

Multiprogramming keeps multiple programs in memory so the CPU always has something to execute when one process is waiting on I/O, improving utilization on a single CPU. Multitasking extends this idea to give the illusion of several programs running 'simultaneously' via time-slicing, often in an interactive context. Multiprocessing refers to a system with multiple physical CPUs/cores actually executing instructions in parallel at the same instant.

Computer Networks

Q47. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Q48. What is a subnet mask, and how do you calculate the number of usable hosts in a subnet?

Answer:

A subnet mask defines which portion of an IP address is the network part and which is the host part (e.g. 255.255.255.0 means the first 24 bits are network, the last 8 are host). For a /24 subnet, there are $2^8 = 256$ total addresses, minus 2 reserved (network address and broadcast address), giving 254 usable host addresses.

Q49. Explain the seven layers of the OSI model and the role of each layer.

Answer:

The seven OSI layers, bottom to top: **Physical** - raw bit transmission over a medium; **Data Link** - framing and MAC addressing between directly connected nodes; **Network** - logical addressing and routing between networks (IP); **Transport** - end-to-end reliable/unreliable delivery (TCP/UDP); **Session** - managing sessions/connections between applications; **Presentation** - data formatting, encryption, compression; **Application** - the actual protocols apps use (HTTP, FTP, DNS).

OSI Model - 7 Layers



Q50. Explain the difference between a hub, a switch, and a router.

Answer:

A hub is a simple, dumb device that repeats incoming signals to every port (Layer 1), causing collisions and wasted bandwidth. A switch operates at Layer 2, learning MAC addresses and forwarding frames only to the specific port the destination device is on, reducing collisions. A router operates at Layer 3, forwarding packets between different networks based on IP addresses, and is what connects a LAN to the internet.

Q51. Explain the difference between IPv4 and IPv6.

Answer:

IPv4 uses 32-bit addresses (about 4.3 billion possible addresses, written as four decimal octets like 192.168.1.1) and is running out of available addresses. IPv6 uses 128-bit addresses (an astronomically larger space, written in hexadecimal groups like 2001:0db8::1), and also simplifies header processing and has built-in support for features IPv4 needed extensions for, like auto-configuration.

Q52. What happens during TCP connection termination (four-way handshake)?

Answer:

TCP connection termination typically uses a four-way handshake: the initiating side sends a FIN (finish); the other side ACKs it (it may still be sending data); once the other side is also done sending, it sends its own FIN; the initiating side ACKs that FIN, formally closing the connection. Two separate FIN/ACK exchanges are needed because TCP connections are full-duplex - each direction must be closed independently.

Q53. What is the role of DNS, and walk through what happens when you type a URL into a browser.

Answer:

DNS translates a human-readable domain name into an IP address. When you type a URL: the browser checks its cache, then the OS cache, then queries a DNS resolver, which walks the DNS hierarchy (root -> TLD -> authoritative nameserver) if needed to resolve the domain to an IP; the browser then opens a TCP connection (and TLS handshake for HTTPS) to that IP, sends an HTTP request, and renders the response.

Q54. What is the difference between TCP and UDP? Give an example use case for each.

Answer:

TCP is connection-oriented, establishes a session via a handshake, guarantees reliable and ordered delivery through acknowledgments and retransmission, but has more overhead - used for web pages, file transfers, and email. UDP is connectionless, sends packets with no delivery guarantee or ordering, but is faster with lower overhead - used for DNS lookups, video calls, and online gaming where speed matters more than occasional lost packets.

TCP	UDP
Connection-oriented Reliable, ordered delivery Slower (handshake+ack) e.g. HTTP, FTP, Email	Connectionless Best-effort, unordered Faster, lower overhead e.g. DNS, video/voice calls

Q55. What is NAT, and why is it necessary given the limited IPv4 address space?

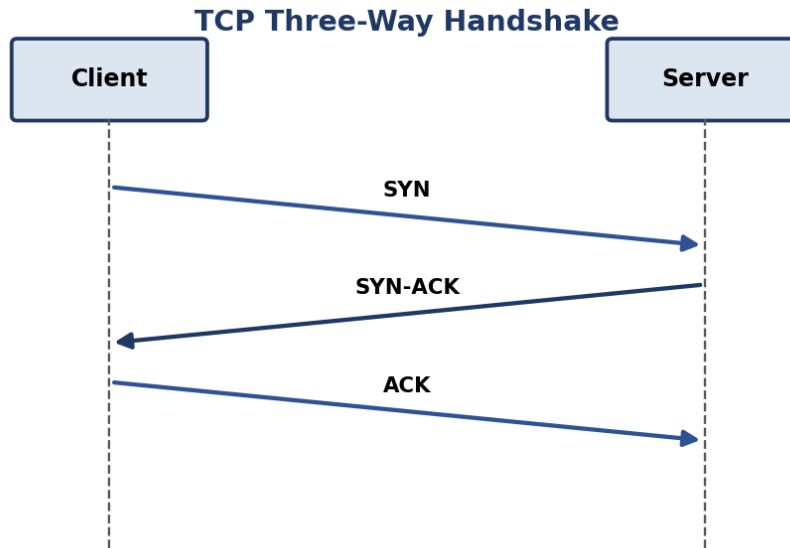
Answer:

NAT (Network Address Translation) allows multiple devices on a private network to share a single public IP address when communicating with the internet, translating private IPs to the public IP (and back for return traffic) at the router/gateway. It's necessary because IPv4's roughly 4.3 billion addresses are far fewer than the number of internet-connected devices worldwide.

Q56. Explain the TCP three-way handshake process for establishing a connection.

Answer:

The three-way handshake establishes a reliable TCP connection: the client sends a SYN (synchronize) packet to the server; the server responds with a SYN-ACK, acknowledging the client's SYN and sending its own; the client responds with an ACK, acknowledging the server's SYN. After this, both sides have confirmed each other's readiness to send/receive, and data transfer begins.



Data Structures & Algorithms

Q57. How would you merge two sorted arrays/linked lists efficiently?

Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in $O(n + m)$ time, which is also the core merge step used inside Merge Sort.

Q58. What is the sliding window technique, and in what type of problems is it useful?

Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k , or longest substring without repeating characters), typically turning an $O(n^2)$ brute force into an $O(n)$ solution.

Q59. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed $O(n \log n)$ time in best, average, and worst cases, is stable, but requires $O(n)$ extra space for merging. Quick Sort also averages $O(n \log n)$ but can degrade to $O(n^2)$ in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q60. Explain how binary search works and derive its time complexity.

Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is $O(\log n)$ - a huge improvement over linear search's $O(n)$ for large sorted datasets.

Q61. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in $O(n)$ time and $O(1)$ space.

Singly Linked List

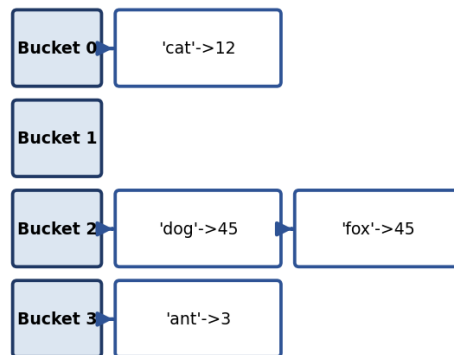


Q62. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average $O(1)$ time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).

Hash Table with Chaining (collision resolution)



Q63. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually $O(1)$, but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an $O(n)$ operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to $O(1)$ amortized.

Q64. Explain topological sorting and where it is used.

Answer:

Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q65. How would you find the kth largest element in an unsorted array efficiently?

Answer:

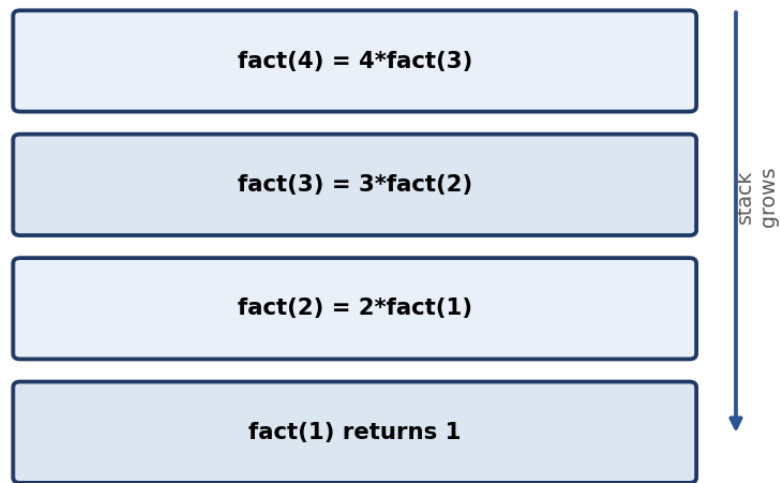
One efficient approach uses a min-heap of size k : iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the k th largest, achieving $O(n \log k)$ time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average $O(n)$ time.

Q66. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.

Call Stack while computing factorial(4)

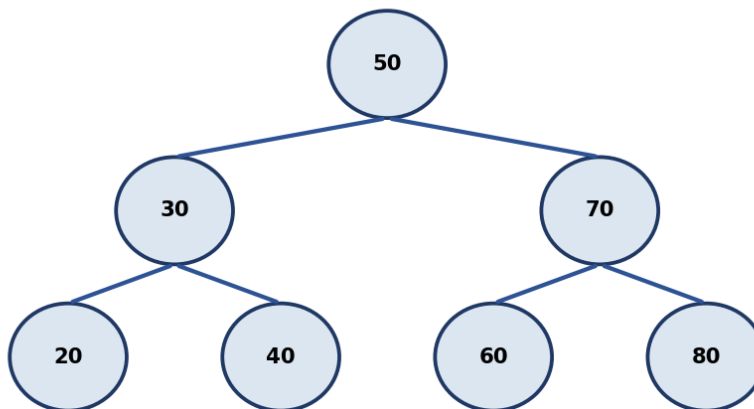


Q67. What is a binary search tree, and what is the time complexity of insertion, deletion, and search in the average and worst case?

Answer:

A Binary Search Tree (BST) is a tree where every node's left subtree contains only smaller values and its right subtree contains only larger values, enabling efficient ordered operations. In a balanced BST, insertion, deletion, and search are all $O(\log h)$ where h is the height (roughly $\log n$ for a balanced tree). In the worst case (a degenerate, list-like tree caused by inserting sorted data without rebalancing), all three operations degrade to $O(n)$.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q68. Explain how you would design an LRU cache and the data structures you'd use to achieve $O(1)$ get/put operations.

Answer:

An LRU cache needs $O(1)$ get and put operations. The standard design combines a hashmap (key $\rightarrow$ pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in $O(1)$; when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in $O(1)$, with the hashmap updated accordingly.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q69. Explain the difference between time complexity and space complexity with examples.

Answer:

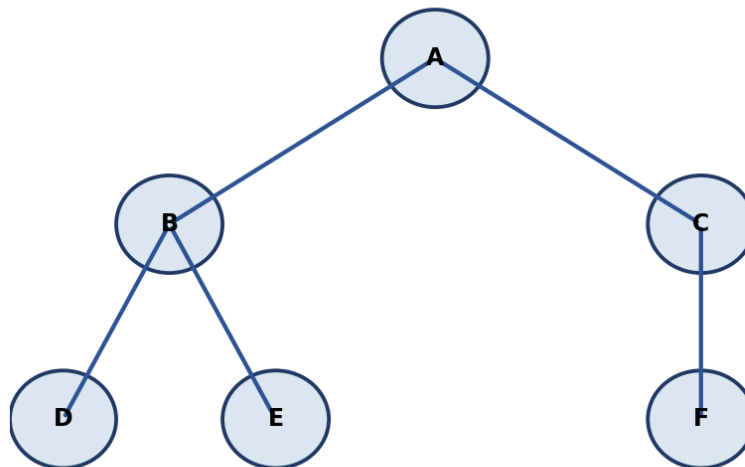
Time complexity measures how an algorithm's runtime grows as input size increases (e.g. $O(n)$ for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. $O(1)$ for an in-place swap vs $O(n)$ for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q70. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F



Q71. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q72. What is backtracking, and how is it different from brute force?

Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Coding Problems

Q73. Write a program to find the maximum subarray sum (Kadane's Algorithm).

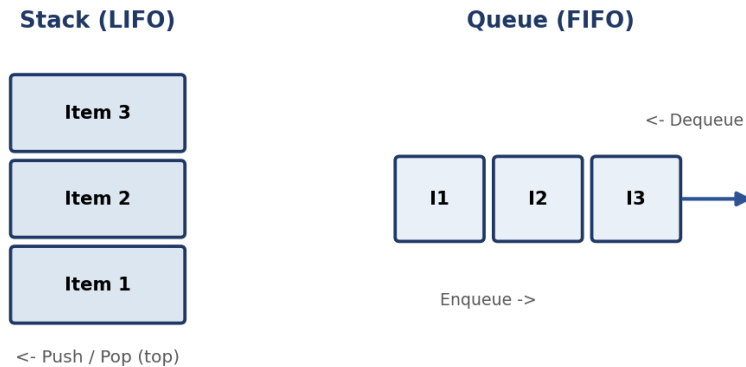
Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. $\text{current_sum} = \max(\text{arr}[i], \text{current_sum} + \text{arr}[i])$), while tracking the overall maximum seen so far. This finds the maximum subarray sum in $O(n)$ time and $O(1)$ space.

Q74. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized $O(1)$ per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.



Q75. Write a program to find the first non-repeating character in a string.

Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in $O(n)$ time and $O(k)$ space, where k is the size of the character set.

Q76. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in $O(n \log n)$ time due to the initial sort.

Q77. Write a program to check if two strings are anagrams of each other.

Answer:

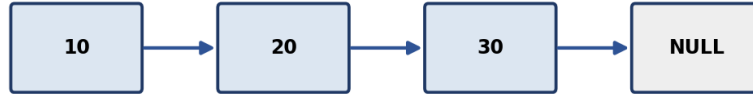
Two common approaches: (1) sort both strings and check if the sorted versions are equal ($O(n \log n)$); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams ($O(n)$ time, $O(1)$ or $O(k)$ space).

Q78. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in $O(n)$ time; iterative uses $O(1)$ extra space while recursive uses $O(n)$ call-stack space.

Singly Linked List



Q79. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return `-1/None`. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



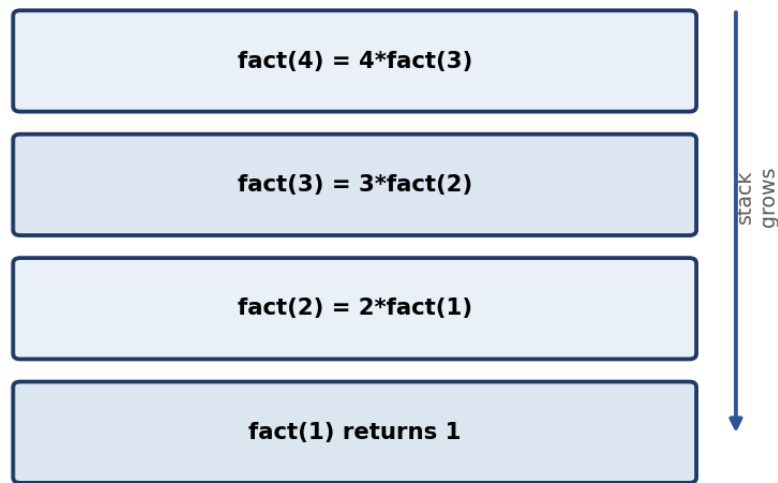
$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q80. Write a program to find the factorial of a number using recursion.

Answer:

Recursive definition: $\text{factorial}(n) = n * \text{factorial}(n-1)$, with a base case $\text{factorial}(0) = 1$ (or $\text{factorial}(1) = 1$). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is $O(n)$; space complexity is $O(n)$ due to the recursive call stack.

Call Stack while computing factorial(4)



Q81. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for $O(1)$ lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in $O(n + m)$ time and $O(\min(n, m))$ space.

Q82. Write a program to find the missing number in an array containing n distinct numbers from 0 to n .

Answer:

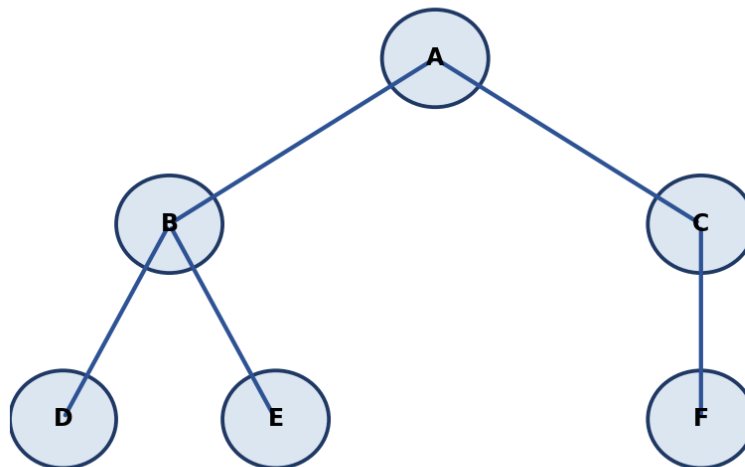
Sum formula approach: the expected sum of numbers 0 to n is $n*(n+1)/2$; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - $O(n)$ time, $O(1)$ space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q83. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in $O(\text{rows} * \text{cols})$ time since each cell is visited a constant number of times.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F

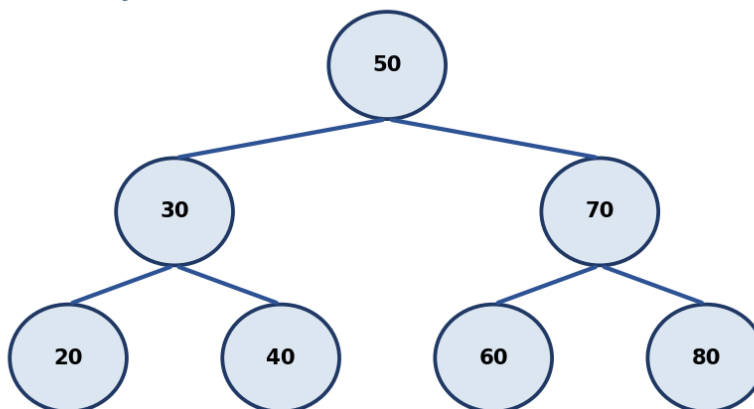


Q84. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q85. Write a program to check whether a given string is a palindrome.

Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in $O(n)$ time and $O(1)$ extra space (ignoring the input itself).

Q86. Write a program to remove duplicates from a sorted array in-place.

Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in $O(n)$ time and $O(1)$ extra space.

Programming Language Fundamentals

Q87. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q88. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. `builder.setName('X').setAge(30).build()`. To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q89. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q90. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Q91. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q92. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q93. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q94. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: `sorted(items, key=lambda x: x.age)`. They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q95. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via `malloc/new`), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q96. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract `Shape` class with an abstract `area()` method but a concrete `describe()` method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a `Circle` class extending `Shape` and implementing `area()`.

Web & Cloud Fundamentals

Q97. What is the difference between HTTP and HTTPS, and how is HTTPS implemented under the hood?

Answer:

HTTP transmits data in plaintext, so it's vulnerable to eavesdropping and tampering in transit. HTTPS secures HTTP by layering it on top of TLS/SSL: a handshake negotiates a shared session key and verifies the server's identity via its certificate, and all subsequent HTTP traffic is encrypted using that session key, protecting confidentiality and integrity even over an untrusted network.

Q98. Explain how a CDN improves website performance for geographically distributed users.

Answer:

A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

Q99. What is a RESTful API, and what makes an API design 'RESTful'?

Answer:

A RESTful API is an API designed around REST (Representational State Transfer) architectural principles: it uses standard HTTP methods (GET, POST, PUT, DELETE) mapped to resource operations, is stateless (each request contains all information needed), uses resource-based URLs (e.g. /users/123), and typically returns representations of resources (often JSON). An API is 'RESTful' to the extent it actually follows these constraints rather than just using HTTP as a transport.

Q100. Explain the difference between IaaS, PaaS, and SaaS with real examples of each.

Answer:

IaaS (Infrastructure as a Service) provides raw virtualized compute/storage/networking that you configure yourself, e.g. AWS EC2 or Azure VMs. PaaS (Platform as a Service) provides a managed platform to build and deploy applications without managing the underlying servers, e.g. Heroku or Azure App Service. SaaS (Software as a Service) delivers a complete, ready-to-use application over the internet, e.g. Gmail or Salesforce - you just use it, with no infrastructure or platform management at all.

Note: 22 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.